```
object Wallet {

  val typeKey = EntityTypeKey[Command]("wallet")      ❶

  ...                                                 ❷

  def apply(walletId: String): Behavior[Command] =
    EventSourcedBehavior[Command, Event, State](      ❸
      PersistenceId(typeKey.name, walletId),
      State(0),
      commandHandler = handleCommands,
      eventHandler = handleEvents)
      ...
      .withRetention(RetentionCriteria
        .snapshotEvery(numberOfEvents = 100, keepNSnapshots = 2))
      .onPersistFailure(
        SupervisorStrategy.restartWithBackoff(
          minBackoff = 10.seconds,
          maxBackoff = 60.seconds,
          randomFactor = 0.1))
  ...
}
```

❶ Adds a key to make the actor sharded

❷ Protocol

❸ Adds EventSourcedBehavior to make the actor persistent

Note that the initial balance in the constructor is set to zero with `State(0)`. You also add snapshot retention and restart with back-off. Depending on the specifics of the betting office, these parameters may vary, but that's an optimization problem. The settings are good enough for now.

In the construction of this actor, in the `apply` method, you start with a balance of zero and then redirect the processing of commands and events to their respectively named methods, `handleCommands` and `handleEvents`. The tagging, snapshotting, and restarting strategies are like those you saw in chapter 9.